



SIMATS

ENGINEERING



Student Name : Devayani P.

Reg no. – 192421082

Course Code - **CSA37**

Course Name - **Software Testing**

Assessment No. – 1

Tool no. – 2



SIMATS
ENGINEERING



SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3

5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3

14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3

23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I Devayani P. (192421082) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Answers :

1. Testing as an Engineering Activity

Random testing may find some defects, but it does not provide proper coverage. Testing should be treated as an engineering activity, where it is planned and carried out systematically.

The team should understand the requirements, prepare test scenarios and design test cases using techniques such as equivalence partitioning, boundary value analysis and decision tables.

For example, if an application accepts marks from 0 to 100, values like -1, 0, 1, 99, 100 and 101 should be tested.

Structured testing gives better coverage and helps detect defects that random testing may miss.

2. Well-Defined Testing Process in Banking

A banking system needs a proper testing process because even a small defect can affect customer transactions.

The process should include requirement analysis, test planning, test case design, test execution, defect reporting, retesting and regression testing.

For example, during fund transfer, testers should check successful transfers, insufficient balance, invalid account numbers and transaction limits.

Following the same process for every release reduces missed test cases and improves the reliability and quality of the banking system.

3. Testing as a Structured SDLC Process

Testing should be carried out throughout the SDLC rather than only after development is completed.

During the requirement phase, testers identify unclear requirements. During development, unit testing is done. During integration, communication between modules is tested. Finally, system and acceptance testing check the complete application.

For an e-commerce system, login, product search, cart and payment should first be tested separately and then together.

Following a structured process helps detect defects early and reduces integration problems.

4. Error, Defect and Failure

An error is a mistake made by a person. A defect is the fault introduced into the software because of that mistake. A failure occurs when the software produces an incorrect result.

For example, a developer misunderstands a password requirement. This is an error. The incorrect password-validation code is the defect. If a user can log in using the wrong password, it becomes a failure.

The relationship can be remembered as:

Error → Defect → Failure

Understanding these terms helps testers report problems clearly and developers fix them correctly.

5. Early Defect Detection

One important testing principle is early testing. It means testing should start as early as possible in the software development process.

Testers can review requirements and design documents before coding begins. This helps identify mistakes before they become actual software defects.

For example, if a banking requirement does not mention the maximum transfer amount, the tester can identify the missing information during requirement review.

Finding defects early is cheaper and easier than fixing them after deployment. Therefore, early testing improves quality and reduces development cost.

6. Tester Involvement in Requirement Analysis

Testers should be involved during requirement analysis instead of joining only after coding is finished.

They can check whether requirements are clear, complete, consistent and testable. They can also identify missing conditions and prepare initial test scenarios.

For example, in a healthcare system, a tester may ask what users are allowed to access a patient's medical information.

Early involvement prevents misunderstandings and reduces defects during development. It also improves communication between testers, developers and business teams.

7. Defects Caused by Unclear Requirements

Defects can originate at different stages of the software lifecycle. If requirements are unclear, developers may understand and implement them differently.

For example, if a payroll requirement does not clearly explain how overtime should be calculated, the developer may use the wrong formula.

The incorrect salary calculation is then a requirement-related defect.

Clear requirements, reviews, examples and acceptance criteria can prevent such problems. Therefore, good requirements are important for producing good-quality software.

8. Boundary Value Defect

If a system crashes when boundary values are entered, it can be considered a functional defect, because the system is not handling input as expected.

For example, if an application accepts age from 18 to 60, the tester should check 17, 18, 19, 59, 60 and 61.

This technique is called Boundary Value Analysis (BVA).

Testing only normal values like 30 or 40 may miss such defects. Boundary testing helps find problems at the limits and improves test coverage.

9. Defect Repository for Better Test Design

A defect repository stores information about defects found in previous releases. It should be used not only for tracking but also for improving future testing.

For example, if payment-related defects occurred repeatedly, testers can create more payment-related test cases in the next release.

The repository can also show which modules have more defects and what types of problems occur frequently.

Using this information helps testers focus on risky areas, improve regression testing and prevent repeated defects.

10. ATM Functional Defect

If an ATM handles a withdrawal incorrectly, it is a functional defect because the system is not performing the required operation correctly.

For example, if a customer withdraws ₹2,000 from an account containing ₹10,000, the ATM should dispense ₹2,000 and the balance should become ₹8,000.

Testers should check successful withdrawals, insufficient balance, invalid amounts and withdrawal limits.

Proper functional, boundary and negative testing could have detected the problem before the ATM system was released.

11. Defect Clustering

Defect clustering means that a large number of defects are often found in a small number of modules.

For example, in an e-commerce application, the payment and checkout modules may contain more defects because they have complex logic.

Therefore, testers should give more attention to modules with a history of defects, high complexity or high business risk.

This helps the team use limited testing time effectively and improves defect detection.

12. Performance Defect

If a web application becomes very slow when many users access it at the same time, it is a performance defect.

It may be caused by slow database queries, insufficient server resources or inefficient code.

Testers can use load testing, stress testing and performance monitoring to identify the problem.

Correct classification helps the development team understand the type of problem and choose an appropriate solution.

13. Using Historical Defect Data for Regression Testing

Historical defect data can be very useful when preparing regression tests.

Testers can identify modules that had many defects in previous releases and give them higher priority.

For example, if the payment module frequently failed earlier, its old failed test cases should be included in the regression suite.

Repeated defects can also be converted into automated tests. This saves time, improves coverage and reduces the chance of old defects returning.

14. Tester Role in Integration Testing

Testers help reduce integration defects by checking how different modules communicate with each other.

They should be involved early in discussions with developers to identify possible interface problems, missing data and incorrect assumptions.

For example, in an e-commerce system, a tester can verify whether the payment module correctly sends the payment status to the order module.

Early tester involvement improves communication and helps identify integration problems before they become difficult to fix.

15. Defect Prevention

When the same defects appear in every release, the team should focus on defect prevention rather than simply fixing each defect.

The team should perform root-cause analysis to understand why the defect keeps occurring. Code reviews, requirement reviews, better test cases and developer training can then be introduced.

For example, if incorrect calculations repeatedly occur, the team can review the calculation requirements and add automated calculation tests.

Defect prevention reduces repeated problems and improves the quality of future releases.

16. Standardizing Mobile Application Testing

Frequent crashes caused by inconsistent testing show that the team needs a common testing strategy.

The team should use standard test plans, test case formats, defect-reporting methods and regression test suites.

Testing should cover functional, negative, compatibility, performance and recovery scenarios.

For example, a mobile app should be tested on different devices, operating systems and network conditions.

Standardized testing makes the process consistent and improves application reliability.

17. Testing and Debugging

Testing and debugging are different activities.

Testing is mainly used to find failures or defects, while debugging is done to find the cause of a defect and fix it.

For example, a tester finds that a payment of ₹1,000 is displayed as ₹10,000. The tester reports the defect. The developer then checks the code and finds an incorrect calculation.

So, testing finds the problem, while debugging finds and fixes its cause. Both are important for software quality.

18. Cross-Browser Testing

An online shopping system should be tested on commonly used browsers such as Chrome, Firefox, Edge and Safari.

Important functions like login, product search, shopping cart, checkout and payment should be checked across different browsers and devices.

Different browser versions, CSS rendering and JavaScript support can cause inconsistent behaviour.

Tools such as Selenium or Playwright can help automate cross-browser testing. This improves compatibility and gives customers a consistent experience.

19. Validation in a Logistics Application

If a logistics application produces inconsistent outputs, one possible reason is poor input validation.

Testers should check valid, invalid, empty, incorrect and boundary inputs. For example, a delivery date should not accept an invalid date or a date earlier than the order date.

Negative testing and boundary value analysis can help identify these problems.

Proper input validation prevents incorrect data from entering the system and improves the reliability of the application.

20. Systematic Test Design

Random testing does not guarantee that all important conditions are covered. Therefore, testing should be treated as an engineering activity.

Testers should first study the requirements and then design test cases using techniques such as equivalence partitioning, boundary value analysis and decision tables.

Each test case should contain clear inputs, steps and expected results.

Systematic test design makes testing repeatable and improves defect detection because important conditions are deliberately tested.

21. Data Consistency in Healthcare System

Data inconsistency between healthcare modules can occur because of incorrect database updates, integration errors or incorrect data mapping.

For example, a patient's address may be updated in the registration module but remain unchanged in the billing module.

Testers should perform integration testing and database validation to check whether data is correctly transferred between modules.

Testing create, update and retrieve operations helps maintain accurate and consistent patient information.

22. Tester Role Across SDLC

Testers should be involved throughout the SDLC.

During requirements, they check clarity and testability. During design, they identify risks. During development, they prepare test cases and support integration testing. During system testing, they execute functional and non-functional tests.

Early involvement helps identify problems before they become expensive to fix.

It also improves communication between developers, testers and business teams, resulting in better-quality software.

23. Testing for Peak Usage

If a system fails during peak usage, it may be because the team tested only normal conditions.

Testers should perform load, stress, volume and endurance testing.

For example, if an online learning system normally has 500 users but may have 10,000 users during examinations, testing should simulate that situation.

Including performance scenarios in the test plan helps find bottlenecks before production and improves system reliability.

24. Defect Documentation and Repository

A proper defect repository helps the team record and track software issues in one place.

Each defect should contain a defect ID, title, steps to reproduce, expected result, actual result, severity, priority and status.

For example, instead of reporting “Payment not working,” the tester should explain exactly what steps cause the payment failure.

Good documentation helps developers reproduce and fix the problem quickly. It also provides useful historical information for future testing.

25. Improving Banking Transaction Testing

Repeated banking failures show that testing needs to be more structured.

Testers should cover successful transactions as well as insufficient balance, invalid accounts, transaction limits, duplicate transactions, network failures and session timeouts.

Techniques such as boundary value analysis, equivalence partitioning and decision tables can be used.

After fixing defects, retesting and regression testing should be performed. This structured approach improves reliability and reduces transaction failures.

26. Testing Edge Cases

Testing only normal inputs can cause important defects to be missed. Testers should deliberately include edge and boundary conditions.

For example, if a system allows transfers between ₹100 and ₹50,000, values such as ₹99, ₹100, ₹101, ₹49,999, ₹50,000 and ₹50,001 should be tested.

Boundary Value Analysis is useful for finding these problems.

Including edge cases improves test coverage and helps identify failures that occur only at the limits of the system.

27. Process-Based Testing for Consistency

Testing across modules should follow a common process so that one module is not tested much better than another.

The organization can use standard test case templates, common defect-reporting formats, checklists and test execution procedures.

For example, login, payment and order modules can all follow:

Planning → Test Design → Execution → Defect Reporting → Retesting → Regression Testing

Standardization reduces missed cases and improves the overall quality of the software.

28. Input Validation and Robustness

A web application should properly handle unexpected and invalid inputs.

Testers should check empty values, special characters, incorrect formats, very large values, negative values and unexpected data.

For example, if a phone-number field accepts only 10 digits, testers should check 9 digits, 11 digits, letters and empty input.

Negative testing and boundary value analysis help find these problems. Proper validation prevents crashes and makes the application more robust.

29. Reducing Defect Leakage

Defect leakage means defects that were not detected during testing but were discovered after the software was released.

Common reasons include incomplete test coverage, poor requirements, insufficient regression testing and unrealistic test scenarios.

To reduce leakage, the team should use risk-based testing, requirement traceability, regression testing, boundary testing and realistic test environments.

Production defects should also be analysed and added to the regression suite. This reduces repeated leakage and improves software quality.

30. Early Defect Detection

Poor testing strategies can allow defects to remain hidden until later stages, where fixing them becomes more expensive.

The team should start testing early by reviewing requirements and designs before coding. After development, unit, integration, system and regression testing should be performed.

For example, finding an incorrect requirement during review is much easier than discovering it after the whole application has been developed.

Therefore, early and systematic testing reduces rework, saves time and cost, and improves overall software quality.